PBCMT404 : DATABASE SYSTEMS
MINI PROJECT REPORT

“Online Blood Bank and Donor Management System”

Using Python and SQLite

(2025-2026 ACADEMIC YEAR)

B.Tech Semester IV

SREE BUDDHA COLLEGE OF ENGINEERING
(AUTONOMOUS)

PATTOOR P.O., NOORANAD , ALAPPUZHA , KERALA

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

PREPARED BY

ADITHYA [Roll No.]

CONTENTS

CHAPTERS	CONTENTS	PAGE No
Chapter 1	Introduction	1
Chapter 2	Methodology	2-4
Chapter 3	Implementation	5-9
Chapter 4	Results	10-17
Chapter 5	Conclusion	18

CHAPTER - 1
INTRODUCTION

The Online Blood Bank and Donor Management System (OBBDMS) is a database-driven web application designed to manage donor records, hospital blood stock, blood requests, donation records and donation appointments. In a traditional blood bank, these records are often maintained in paper registers, spreadsheets and disconnected hospital systems. Staff must reconcile donor details, available units, pending requests and donation history before they can answer a simple question: does a hospital currently hold usable units of a given blood group?

That manual approach creates duplicated records, stale stock counts and requests whose status is known only to the person who received the phone call. Donors likewise have no reliable way to check when they last donated or when they become eligible to donate again. The project addresses these problems by centralising the core data in one relational database and giving each role a focused interface for the work it needs to perform.

The system is implemented using Python 3.10, Flask 3.1 and SQLite. Flask provides the application layer and route-level access control, Jinja templates and a custom CSS interface provide the presentation layer, and SQLite stores the relational data. Two roles are supported. Donors register, sign in, check their 90-day eligibility, book a donation slot, search for blood and raise requests. Administrators manage hospitals, inventory, requests, donations and donor records from a dispatch-board interface.

From a database-systems perspective, the project applies relational schema design, primary and foreign keys, uniqueness constraints, check constraints, indexes and parameterized SQL. The schema contains six related tables: users, hospitals, inventory, requests, donations and appointments. Inventory is unique per hospital and blood group, requests and donations reference both a user and a hospital, and appointments reference a donor and a hospital. These constraints keep the data consistent when records are inserted or updated through the application.

The primary objectives of the project are to centralise donor and stock data, provide public availability search, compute donor eligibility from donation history, enforce a controlled request lifecycle, protect state-changing operations with authentication and CSRF tokens, and render the same relational data through a usable desktop and mobile interface. The system is deliberately scoped as an academic prototype with synthetic data; it does not verify clinical compatibility, screen blood or replace hospital procedures.

The report follows the PBCMT404 mini-project structure. Chapter 2 explains the requirement analysis, system design, architecture and tools. Chapter 3 presents the database schema and the implementation modules. Chapter 4 shows the working interface and the results of the exercised workflows. Chapter 5 concludes the report and identifies the limitations of the prototype.

CHAPTER - 2
METHODOLOGY

2.1 Requirement Analysis

The requirements were identified from the daily work of a small blood bank. The following functional requirements shaped the design:

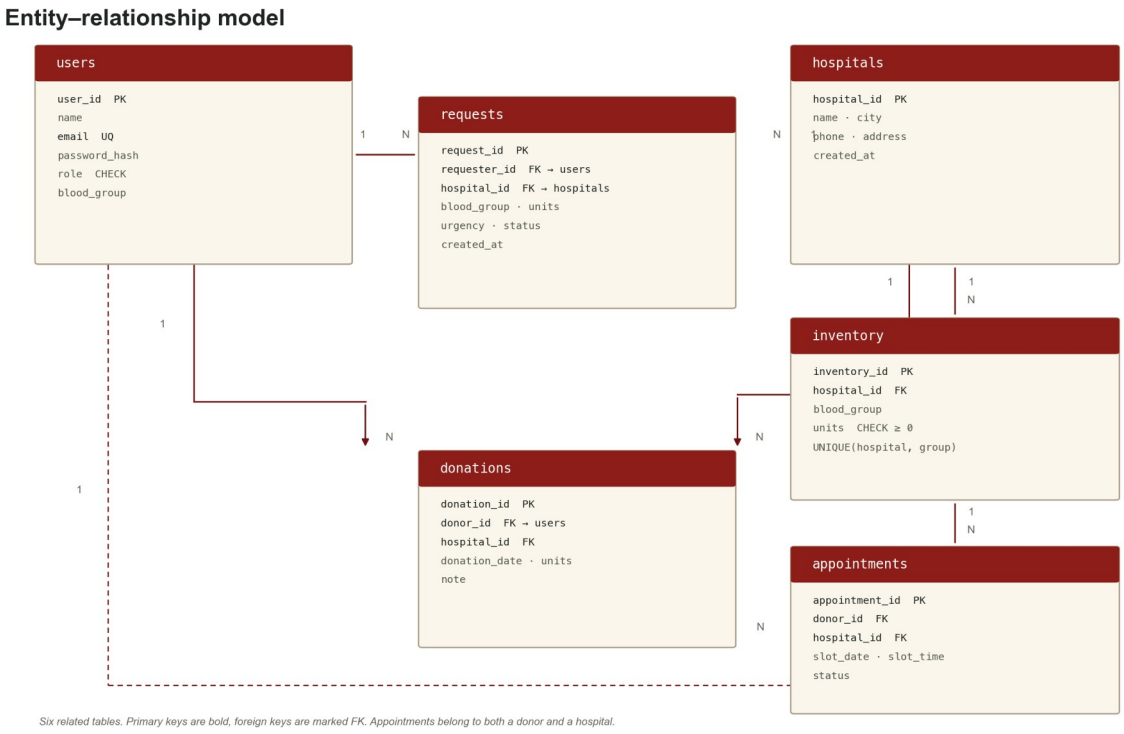
- Public availability search by blood group and city without requiring a login.
- Donor registration and secure login with email and password.
- Donor dashboard with last donation, 90-day eligibility, donation history, appointment booking and personal request history.
- Administrator dashboard with stock strips, critical groups, pending requests, recent donations and hospital totals.
- Hospital and inventory management with one stock line per hospital and blood group.
- Request lifecycle: Pending, Approved, Completed and Cancelled, with stock deducted only on valid completion.
- Donation logging, hospital management and donor record search.
- Role-based access control, CSRF protection and server-side validation on every write.

2.2 System Design

The design follows an entity-relationship model. The entities and relationships are:

- users: donors and administrators, with a unique email and a role constrained to donor or admin.
- hospitals: blood-bank sites with name, city, phone and address.
- inventory: one row per hospital and blood group, with a non-negative unit count and a UNIQUE pair (hospital_id, blood_group).
- requests: raised by a user for a hospital, blood group, unit count, urgency and status.
- donations: completed donations by a donor at a hospital, used for history and eligibility.
- appointments: booked donation slots for a donor at a hospital, with date, time and status.

A user can raise many requests, make many donations and book many appointments. A hospital can hold many inventory lines, receive many requests and record many donations. Each of these one-to-many relationships is enforced with a foreign key, and SQLite foreign-key enforcement is enabled on every connection.



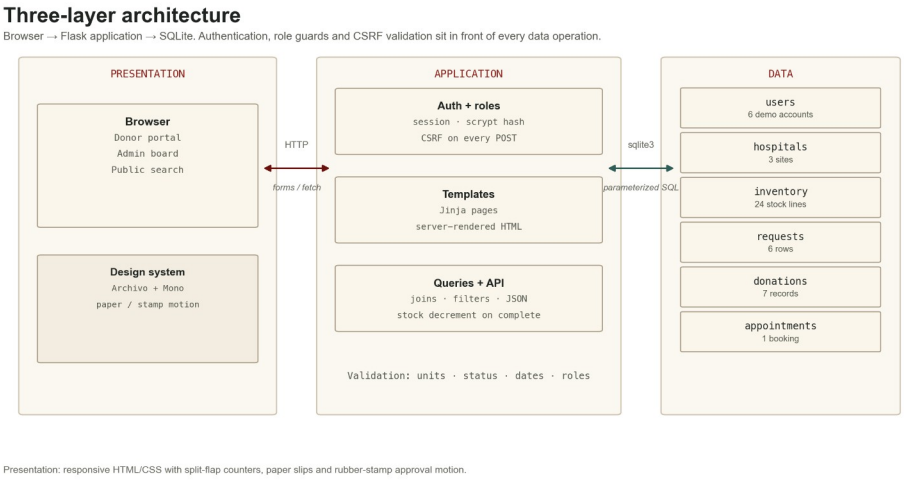
fig(2.1) Entity–Relationship Diagram

2.3 Architecture

The application uses a three-tier architecture:

- Presentation tier: server-rendered Jinja templates, a custom CSS design system and a small amount of JavaScript for status updates, counters and motion.
- Application tier: Flask routes, authentication decorators, CSRF validation, request validation and parameterized database queries.
- Database tier: SQLite with six related tables, foreign-key constraints, check constraints, unique constraints and search indexes.

The browser sends an HTTP request to Flask. The application verifies the session, role and CSRF token where required, runs a parameterized query and renders a template with the result. The same route structure powers the desktop and mobile layouts; the responsive behaviour is implemented in CSS rather than a separate mobile application.



fig(2.2) Three-Tier Architecture

2.4 Tools and Technologies Used

- Programming language: Python 3.10.
- Web framework: Flask 3.1.3.
- Database: SQLite 3, with a MySQL-portable relational schema.
- Templates: Jinja2.
- Frontend: HTML5, custom CSS and vanilla JavaScript.
- Security: Werkzeug password hashing, signed sessions and CSRF tokens.
- Testing: Flask test client and Google Chrome through Playwright.
- Diagrams: Matplotlib; screenshots: Playwright at 1440×900 and 390×844.

CHAPTER - 3
IMPLEMENTATION

3.1 Database Implementation

The database is created automatically on first run. The schema enforces primary keys, foreign keys, check constraints and a unique constraint on inventory. The statements below define the six tables and their indexes.

```
CREATE TABLE users (
  user_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('donor','admin')),
  phone TEXT, city TEXT, blood_group TEXT,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE hospitals (
  hospital_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  city TEXT NOT NULL,
  phone TEXT, address TEXT,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE inventory (
  inventory_id INTEGER PRIMARY KEY AUTOINCREMENT,
  hospital_id INTEGER NOT NULL,
  blood_group TEXT NOT NULL,
  units INTEGER NOT NULL DEFAULT 0 CHECK (units >= 0),
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),
  UNIQUE (hospital_id, blood_group),
  FOREIGN KEY (hospital_id) REFERENCES hospitals(hospital_id)
);

CREATE TABLE requests (
  request_id INTEGER PRIMARY KEY AUTOINCREMENT,
  requester_id INTEGER NOT NULL,
  hospital_id INTEGER NOT NULL,
  blood_group TEXT NOT NULL,
  units INTEGER NOT NULL CHECK (units > 0),
  urgency TEXT NOT NULL DEFAULT 'Routine',
  note TEXT,
  status TEXT NOT NULL DEFAULT 'Pending'
    CHECK (status IN ('Pending','Approved','Completed','Cancelled')),
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (requester_id) REFERENCES users(user_id),
  FOREIGN KEY (hospital_id) REFERENCES hospitals(hospital_id)
);

CREATE TABLE donations (
  donation_id INTEGER PRIMARY KEY AUTOINCREMENT,
  donor_id INTEGER NOT NULL,
  hospital_id INTEGER NOT NULL,
  donation_date TEXT NOT NULL,
  units INTEGER NOT NULL DEFAULT 1 CHECK (units > 0),
  note TEXT,
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (donor_id) REFERENCES users(user_id),
```

```
  FOREIGN KEY (hospital_id) REFERENCES hospitals(hospital_id)
);

CREATE TABLE appointments (
  appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
  donor_id INTEGER NOT NULL,
  hospital_id INTEGER NOT NULL,
  slot_date TEXT NOT NULL,
  slot_time TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'Booked'
    CHECK (status IN ('Booked','Kept','Missed','Cancelled')),
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (donor_id) REFERENCES users(user_id),
  FOREIGN KEY (hospital_id) REFERENCES hospitals(hospital_id)
);

CREATE INDEX idx_inventory_group ON inventory(blood_group);
CREATE INDEX idx_requests_status ON requests(status);
CREATE INDEX idx_donations_donor ON donations(donor_id);
```

The schema holds one fact per table: a hospital holds stock, a user raises a request, a donor makes a donation and a donor books an appointment. Hospital names are never duplicated inside inventory, and donor details are never duplicated inside requests or donations.

3.2 Database Connection and Query Helpers

A SQLite connection is opened per Flask application context. Foreign-key enforcement is switched on explicitly because SQLite does not enable it by default. A teardown hook closes the connection at the end of the request.

```
def get_db():
    if "db" not in g:
        os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
        g.db.execute("PRAGMA foreign_keys = ON")
    return g.db

@app.teardown_appcontext
def close_db(_exc):
    db = g.pop("db", None)
    if db is not None:
        db.close()

def query(sql, args=(), one=False):
    cur = get_db().execute(sql, args)
    rows = cur.fetchall()
    cur.close()
    return (rows[0] if rows else None) if one else rows

def execute(sql, args=()):
    db = get_db()
    cur = db.execute(sql, args)
    db.commit()
    last_id = cur.lastrowid
    cur.close()
    return last_id
```

3.3 Authentication, Roles and CSRF

Registration validates the email, password length, blood group and required fields, hashes the password with Werkzeug and inserts the user with the donor role. Login verifies the hash and

stores the user id and role in the signed session. The admin_required decorator returns HTTP 403 when a signed-in donor attempts to open an administrator route.
Every POST is guarded by a before_request hook. The token is generated per session, embedded in forms and in a meta tag for fetch calls, and compared with secrets.compare_digest. A missing or stale token is rejected before the route can touch data.

```
@app.before_request
def csrf_protect():
    if request.method == "POST":
        sent = (request.form.get("csrf_token")
                or request.headers.get("X-CSRF-Token"))
        if not sent or not secrets.compare_digest(sent, session.get("csrf", "")):
            if request.path.startswith("/api/"):
                return jsonify({"error": "Invalid CSRF token."}), 400
            abort(400, description="Invalid CSRF token.")

def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if not session.get("user_id"):
            flash("Please sign in to continue.", "error")
            return redirect(url_for("login"))
        return view(*args, **kwargs)
    return wrapped

def admin_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if session.get("role") != "admin":
            abort(403)
        return view(*args, **kwargs)
    return wrapped
```

3.4 Request Lifecycle and Stock Consistency

Status changes are not accepted as free text. The API defines the permitted next state for each current state and returns HTTP 409 for an invalid transition. Completion reads the inventory row for the request's hospital and blood group, refuses the transition if there are not enough units, and only then decrements the inventory and marks the request Completed.

```
allowed = {
    "Pending":   {"Approved", "Cancelled"},
    "Approved":  {"Completed", "Cancelled"},
    "Completed": set(),
    "Cancelled": set(),
}

if new_status not in allowed.get(row["status"], set()):
    return jsonify({"error": "Invalid transition."}), 409

if new_status == "Completed":
    inv = query(
        "SELECT * FROM inventory WHERE hospital_id = ? AND blood_group = ?",
        (row["hospital_id"], row["blood_group"]), one=True)
    if not inv or inv["units"] < row["units"]:
        return jsonify({"error": "Not enough units in stock."}), 409
    execute("UPDATE inventory SET units = units - ? "
            "WHERE inventory_id = ?",
            (row["units"], inv["inventory_id"]))

execute("UPDATE requests SET status = ? WHERE request_id = ?",
        (new_status, request_id))
```

3.5 Availability Search Implementation

Search is built from optional predicates rather than separate queries for every combination. A blood group uses exact equality, while a city uses a parameterized LIKE. The query joins inventory to hospitals so the result can show the site name, city and phone number without duplicating those fields inside the inventory table.

```
sql = """
    SELECT i.blood_group, i.units, i.updated_at,
           h.name, h.city, h.phone
    FROM inventory i
    JOIN hospitals h ON h.hospital_id = i.hospital_id
    WHERE 1=1
"""
if blood:
    sql += " AND i.blood_group = ?"
    args.append(blood)
if city:
    sql += " AND h.city LIKE ?"
    args.append(f"%{city}%")
sql += " AND i.units > 0 ORDER BY i.units DESC, h.name"
rows = query(sql, args)
```

Values are always bound as parameters. An injection-style city string is treated as literal data and returns zero rows rather than altering the query structure.

3.6 Donor Module

The donor dashboard combines eligibility, history, appointments and requests on one page. Eligibility is computed from the most recent donation plus a 90-day cooldown, and the same page exposes appointment booking and personal request submission.

```
def donor_stats(user_id):
    last = query(
        "SELECT donation_date FROM donations WHERE donor_id = ? "
        "ORDER BY donation_date DESC LIMIT 1",
        (user_id,), one=True)
    total = query(
        "SELECT COUNT(*) AS c, COALESCE(SUM(units),0) AS u "
        "FROM donations WHERE donor_id = ?",
        (user_id,), one=True)
    last_date = next_date = None
    days_left = 0
    if last:
        last_date = datetime.strptime(last["donation_date"], "%Y-%m-%d").date()
        next_date = last_date + timedelta(days=90)
        days_left = max(0, (next_date - date.today()).days)
    return {"last_date": last_date, "next_date": next_date,
            "days_left": days_left, "eligible": days_left == 0,
            "count": total["c"], "units": total["u"]}")

@app.route("/book", methods=["POST"])
@login_required
def book_appointment():
    hospital_id = request.form.get("hospital_id", type=int)
    slot_date = request.form.get("slot_date", "")
    slot_time = request.form.get("slot_time", "")
    chosen = datetime.strptime(slot_date, "%Y-%m-%d").date()
    if chosen < date.today():
        flash("Pick today or a future date.", "error")
        return redirect(url_for("donor_dashboard"))
    execute("INSERT INTO appointments (donor_id,hospital_id,slot_date,slot_time) "
            "VALUES (?,?,?,?)",
            (session["user_id"], hospital_id, slot_date, slot_time))
```

3.7 Administrator Module

The administrator board summarises stock by blood group, marks critical groups, and lists pending requests, recent donations and hospital totals. Each pending-request slip joins users and hospitals so it can show the requester, reference, blood group, unit count, urgency and hospital without storing duplicate names.

```
slips = query("""
    SELECT r.*, u.name AS requester, h.name AS hospital, h.city
    FROM requests r
    JOIN users u ON u.user_id = r.requester_id
    JOIN hospitals h ON h.hospital_id = r.hospital_id
    WHERE r.status = 'Pending'
    ORDER BY CASE r.urgency WHEN 'Critical' THEN 0 ELSE 1 END,
             r.created_at
    LIMIT 4
""")
```

Donation logging is a separate administrator action. The route validates the donor, hospital, date and unit count, inserts the donation, reads the donor's blood group, and updates the matching inventory row. If the hospital does not yet hold that group, a new inventory line is created.

```
@app.route("/admin/donations", methods=["GET", "POST"])
@admin_required
def admin_donations():
    if request.method == "POST":
        donor_id = request.form.get("donor_id", type=int)
        hospital_id = request.form.get("hospital_id", type=int)
        units = request.form.get("units", type=int) or 1
        execute("INSERT INTO donations
(donor_id,hospital_id,donation_date,units,note) "
                "VALUES (?,?,?,?,?)",
                (donor_id, hospital_id, donation_date, units, note))
        group = query("SELECT blood_group FROM users WHERE user_id = ?",
                      (donor_id,), one=True)["blood_group"]
        execute("UPDATE inventory SET units = units + ? "
                "WHERE hospital_id = ? AND blood_group = ?",
                (units, hospital_id, group))
    return render_template("admin_donations.html", ...)
```

3.8 Interface Implementation

The interface uses a hospital dispatch-desk metaphor rather than a generic admin template. Cream paper, an oxidized-red rail, ink typography, amber signals and teal approvals form a clear operational surface. Stock is rendered as paper strips with split-flap counters; pending requests appear as paper slips; approval presses a rubber stamp before the slip is filed.

The layout is responsive. At mobile width the fixed rail becomes a compact header with a horizontally scrollable navigation strip, the board collapses to one column, stock strips become two-up and action buttons grow to a 40-pixel minimum height. Wide tables scroll inside their own container. A reduced-motion media query collapses animation and reveals all content immediately, so no state depends on motion.

CHAPTER - 4
RESULTS

The Online Blood Bank and Donor Management System was implemented using Python, Flask, SQLite and a responsive HTML interface. The following screens were captured from the running application with the seeded synthetic demo database. Each screen is followed by the implemented features and the observed result.

4.1 Login Interface

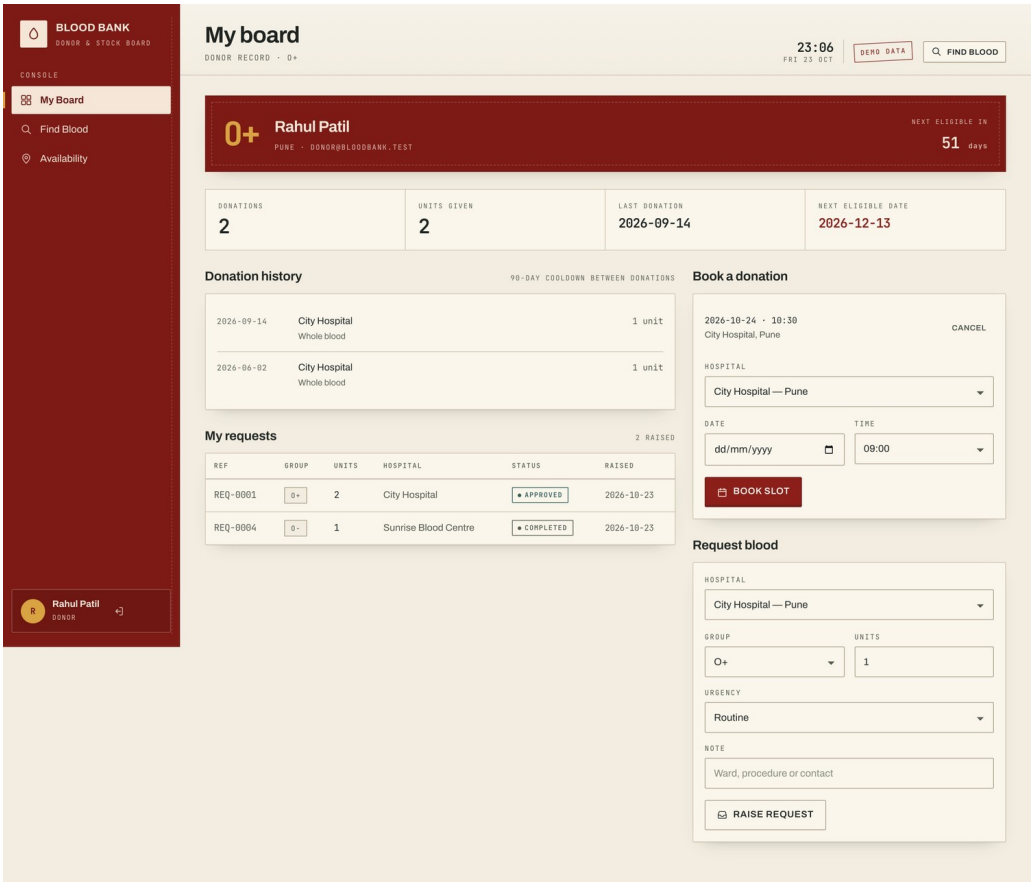


Fig. 4.1 Login Page

This is the entry screen for donors and administrators. The server verifies the password hash, creates a signed session, and redirects the user according to their role.

Features: email and password inputs, CSRF-protected form, password hashing, role-based redirect and clear error messages.

Result: A valid administrator reaches /admin, a valid donor reaches /dashboard, and a missing CSRF token returns HTTP 400.

4.2 Public Landing Page

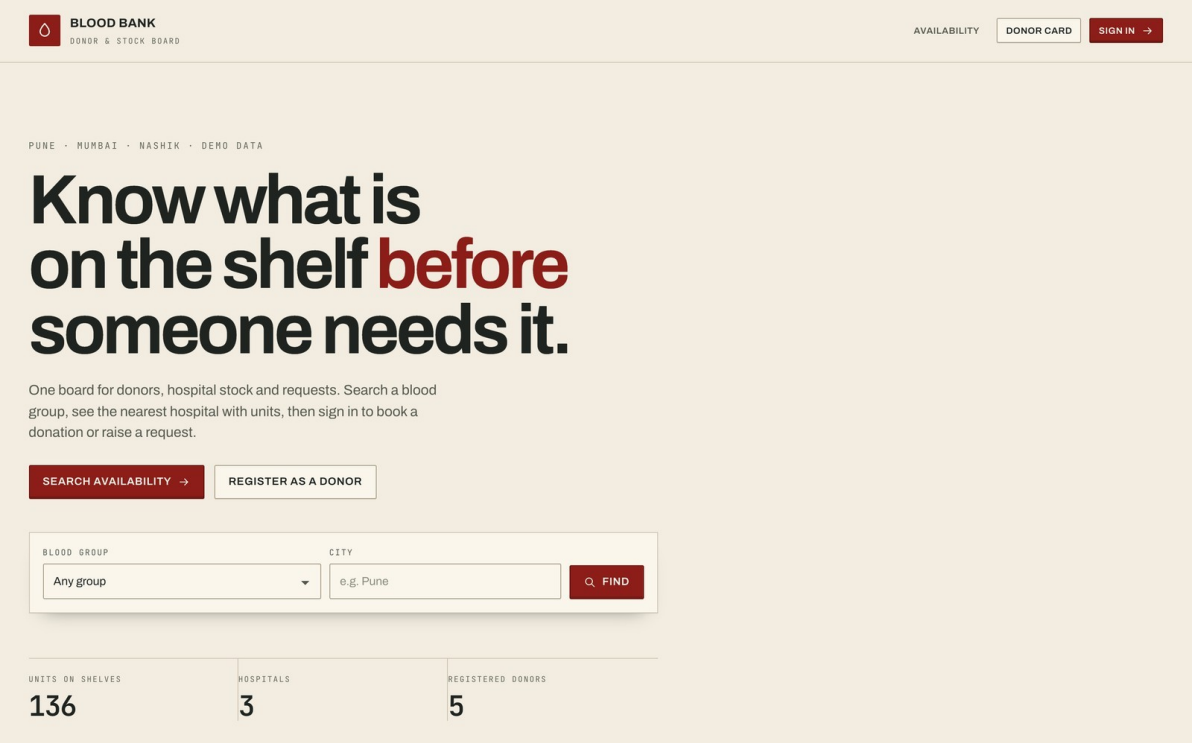


Fig. 4.2 Public Landing Page

The landing page states the problem and gives visitors two immediate actions: search for availability or register as a donor. Group-level stock totals are visible without signing in.

Features: public availability search, donor registration, live stock summary, low-stock signalling and a sign-in route.

Result: A visitor can check whether blood is available in a city before travelling, while donor and administrator actions remain protected.

4.3 Availability Search

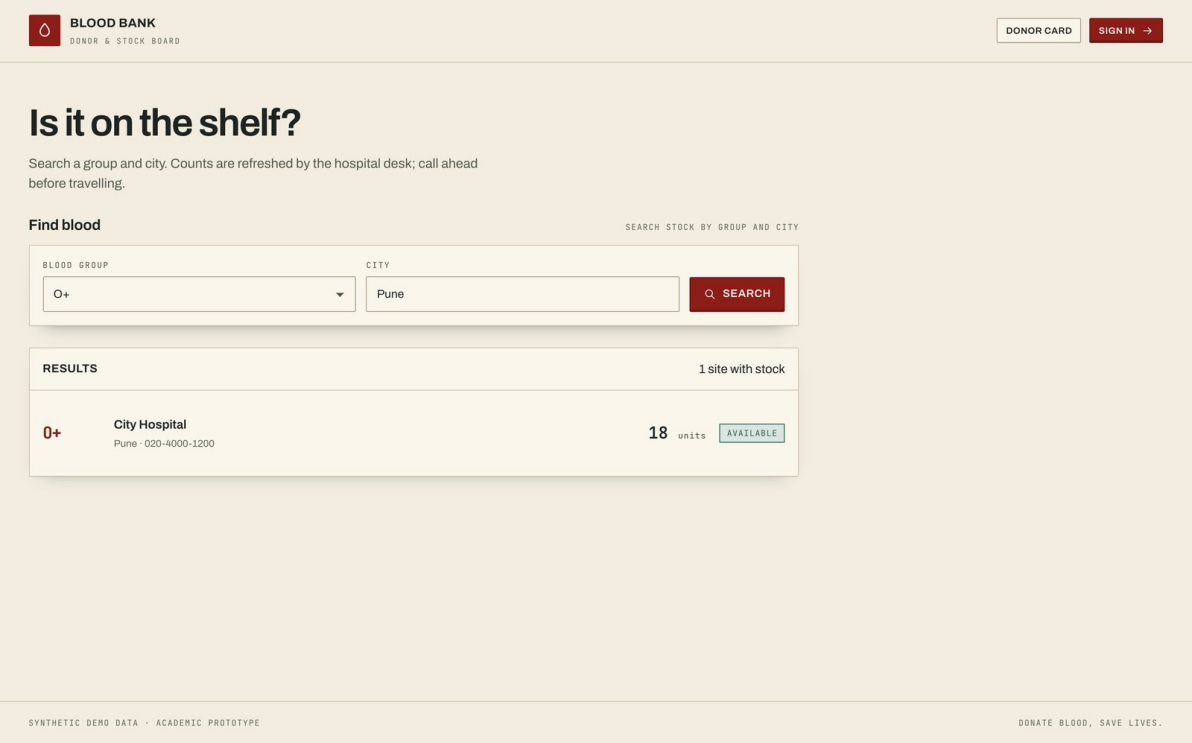


Fig. 4.3 Availability Search for O+ in Pune

The search page returns hospital rows for the selected blood group and city. Each row includes the hospital name, city, phone number and available units.

Features: blood-group filter, city filter, joined hospital data, positive-stock filter and parameterized SQL.

Result: The O+ search in Pune returned City Hospital with its contact number. An injection-style city string returned zero rows and did not expose or alter data.

4.4 Donor Dashboard

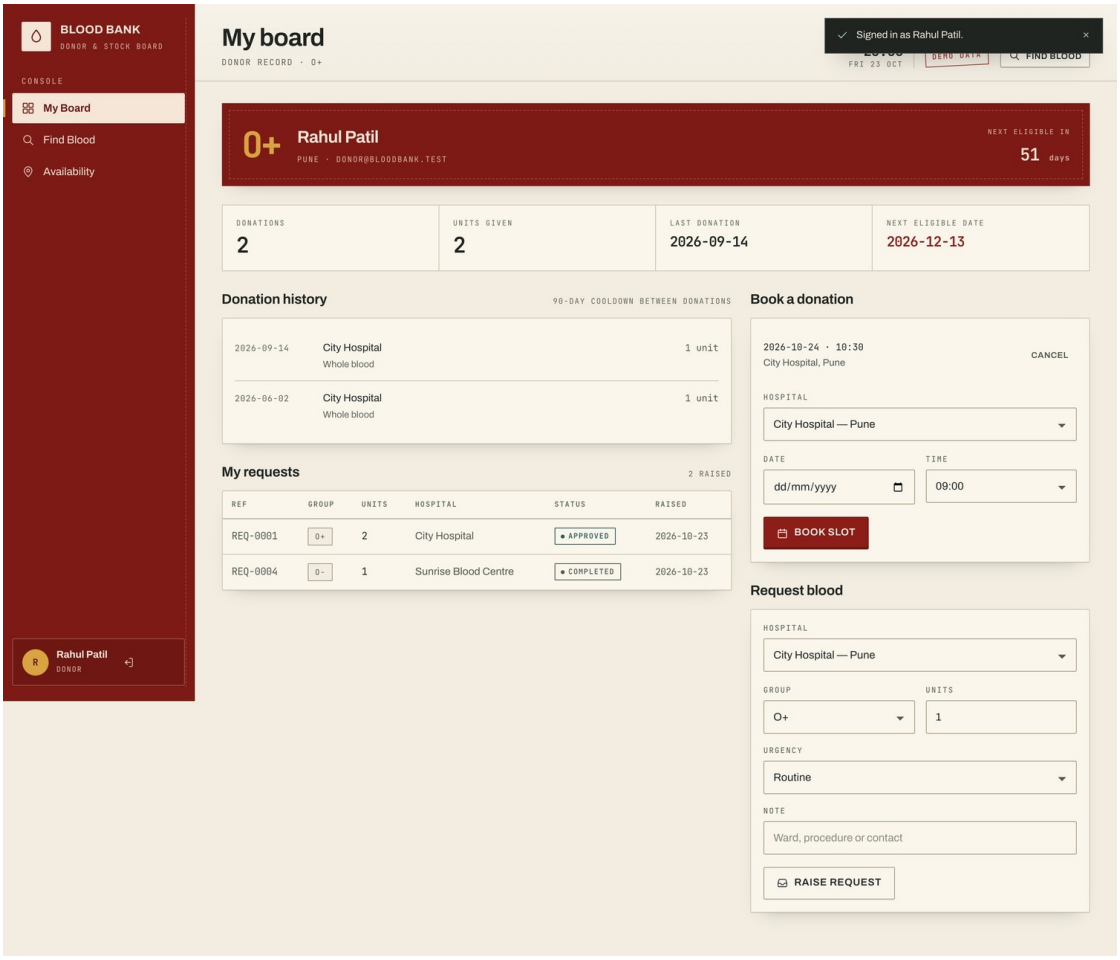


Fig. 4.4 Donor Dashboard

The donor dashboard answers the questions a donor actually asks: when did I last donate, when can I donate again, what have I given and what is happening to my request?

Features: 90-day eligibility strip, donation history, appointment booking, personal request form, request history and appointment cancellation.

Result: A donor can see the next eligible date, book a future slot and review only their own records. Ownership checks prevent cancellation of another donor's appointment.

4.5 Administrator Dashboard

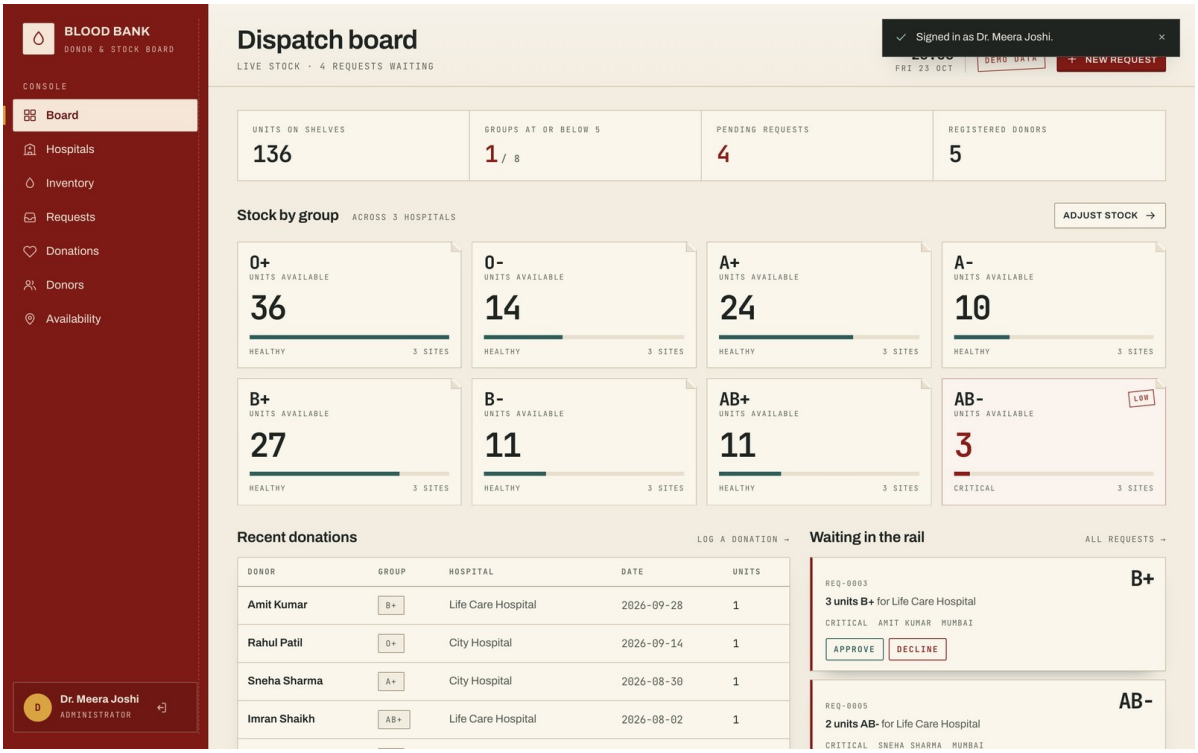


Fig. 4.5 Administrator Dispatch Board

The administrator board is the first viewport after login. Group stock is arranged as split-flap strips, critical groups are marked in red, and pending requests appear as paper slips.

Features: stock strips, critical-group signalling, pending request rail, recent donations, hospital stock totals and top-level statistics.

Result: An administrator can understand stock and pending work at a glance and move directly to the request queue or inventory page.

4.6 Request Management and Status Actions

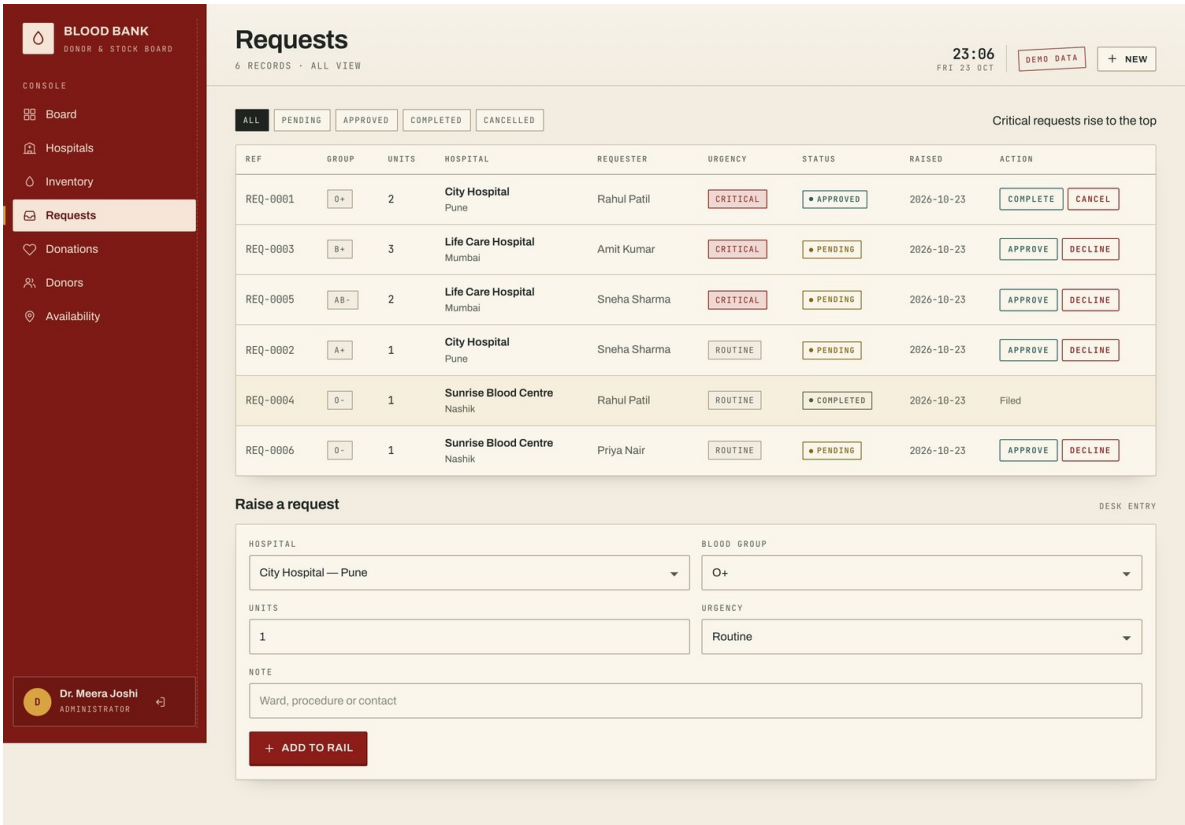


Fig. 4.6 Request Queue with Status Actions

The request page exposes the full queue with status filters. Pending requests offer Approve and Decline; approved requests offer Complete and Cancel. Completing a request is the only action that changes stock.

Features: status filters, permitted actions by state, server-side transition checks, stock validation, confirmation toast and retained terminal rows.

Result: Approval returned HTTP 200, repeat completion returned HTTP 409, and completing a valid request reduced A+ stock from 12 to 11 units.

4.7 Mobile Responsive Interface

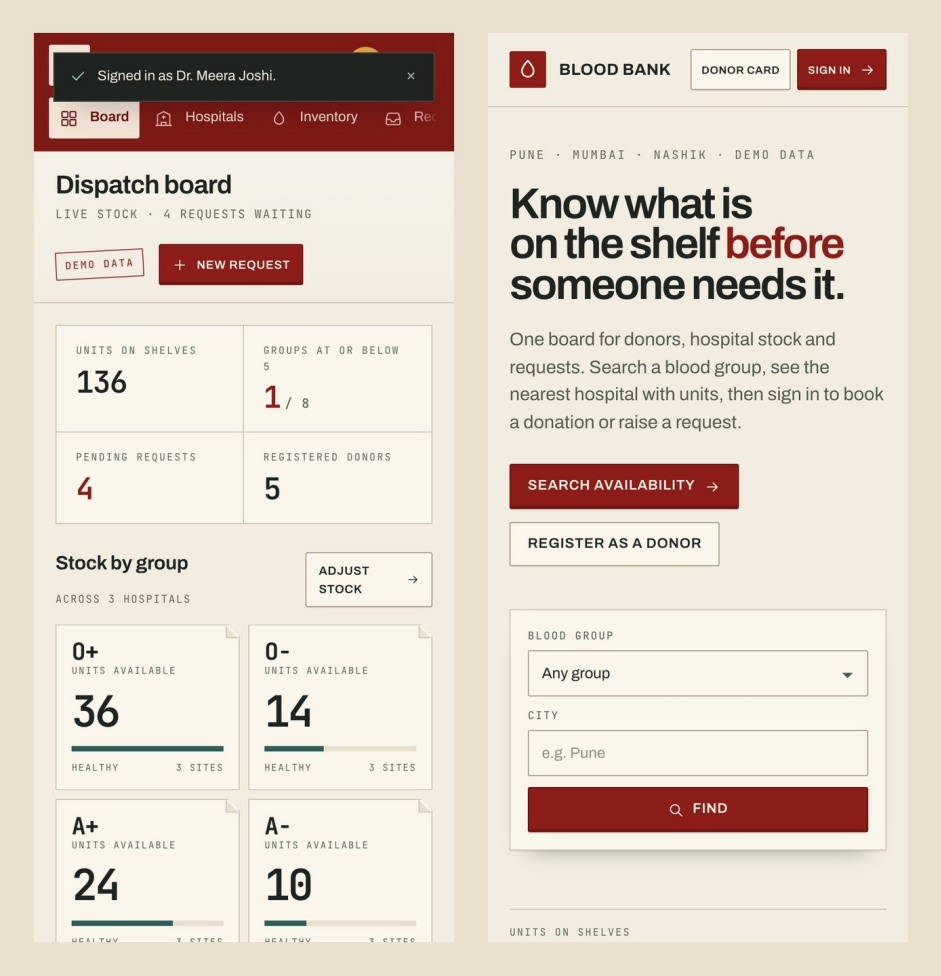


Fig. 4.7 Mobile Layout at 390×844

The mobile layout keeps the same information architecture rather than hiding core actions. The rail becomes a compact header with a scrollable tab strip, the board collapses to a single column and stock strips become two-up.

Features: responsive navigation, one-column board, internally scrolling tables, 40-pixel action targets and a reduced-motion path.

Result: Browser checks found no page-level horizontal overflow at 390 pixels, sampled text met WCAG AA contrast, and reduced motion revealed content immediately.

4.8 Overall System Result

All planned modules were implemented and exercised: registration, login, public search, donor dashboard, administrator board, hospital and inventory management, donation logging, request processing and appointments.

- The six-table relational schema is created automatically and enforces primary keys, foreign keys, uniqueness and check constraints.
- Fourteen functional and security checks were executed, including missing CSRF, role denial, invalid state transitions and injection-style search.
- Browser checks covered desktop and mobile overflow, sampled contrast, reduced motion and loaded assets.
- Stock changed only after a valid completion, and database row counts changed only after a successful request or appointment insert.

The prototype is not an exhaustive penetration test or a clinical validation. It is a working demonstration that database constraints, server-side validation and interface state can be kept consistent through one application.

CHAPTER - 5
CONCLUSION

The Online Blood Bank and Donor Management System demonstrates how a relational database can support a time-sensitive healthcare workflow. The six-table schema captures users, hospitals, inventory, requests, donations and appointments through explicit primary keys, foreign keys, uniqueness rules and check constraints. Each role receives a focused surface: donors find blood, check eligibility and manage their contribution history; administrators see the shelf, clear the request rail and record donations.

From a DBMS perspective, the project applies relational schema design, normalisation, foreign-key integrity, unique and check constraints, indexes and parameterized SQL. The application commits each write, performs stock validation before deducting units, and enforces every write through server-side validation. Authentication, role guards and CSRF protection show that database quality must be paired with application-level security; a correct schema alone does not prevent an unauthorised action.

The interface contributes to the result because it exposes the same relational facts that the schema encodes. Stock appears as ordered units rather than free text, requests move through an explicit state machine rather than undefined statuses, and terminal rows are retained as filed records. The mobile layout keeps the same actions available on a phone, and the reduced-motion path keeps the content usable without animation.

The prototype has clear limits. It uses SQLite, which is adequate for a single-node demonstration but not for concurrent multi-branch operation. It does not verify clinical compatibility, screen donated blood, confirm donor identity or replace hospital procedures. It also lacks email notifications, audit logging, rate limiting and production backup. Future work can migrate the schema to MySQL or PostgreSQL, add versioned migrations, audit trails, notifications, pagination and an automated CI test pipeline.

Overall, the project meets its academic objective: it turns a database exercise into a coherent working application and shows how schema constraints, server-side validation, role-based access and interface craft combine to produce trustworthy behaviour. With the migration and operational work described above, the same structure could serve as the foundation for a larger blood-bank information system.